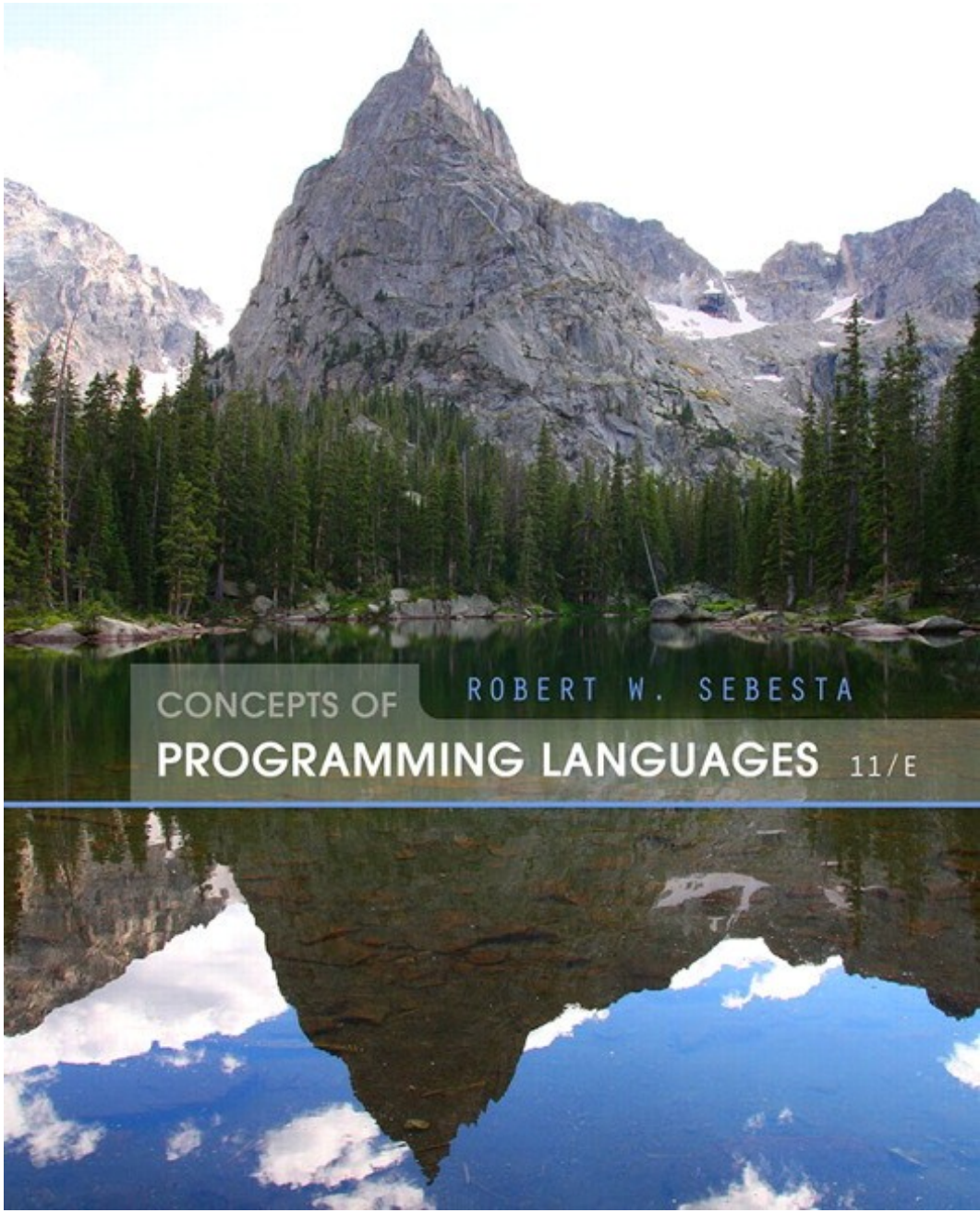
Bölüm 4

Lexical ve Syntax Analizi



ISBN 0-321-49362-1

Bölüm 4 Konular

- 4.1 Giriş
- 4.2 Lexical Analiz
- 4.3 The Parsing Problem
- 4.4 Özyinelemeli İniş Parsing (Recursive-Descent Parsing)
- 4.5 Aşağıdan Yukarıya Parsing (Bottom-Up Parsing)

Copyright © 2015 Pearson. All rights reserved.

4.1 Giriş

- Dil uygulama sistemleri, belirli uygulama yaklaşımından bağımsız olarak kaynak kodunu analiz etmelidir.
- Neredeyse tüm syntax analizleri, kaynak dilin syntax'ının (BNF) formel bir tanımına dayanmaktadır.

Copyright © 2015 Pearson. All rights reserved.

4.1.1 Syntax Analizi

- Bir dil işlemcisinin syntax analizi kısmı neredeyse her zaman iki bölümden oluşur:
  - Sözcük çözümleyicisi *(lexical analyzer)* olarak adlandırılan düşük seviyeli bir parça (matematiksel olarak, düzenli gramer temelli sonlu otomat)
  - *Syntax analizcisi* veya parser (matematiksel olarak, bağlamdan bağımsız qramer (context-free qrammar) veya BNF'ye dayalı bir push-down otomatı) olarak adlandırılan üst düzey bir parça.

Copyright © 2015 Pearson. All rights reserved.

4.1.2 BNF'yi Syntax'i Tanımlamak İçin Kullanmanın Avantajları

- Açık ve özlü bir syntax açıklaması sağlar.
- Parser doğrudan BNF'ye dayalı olabilir.
- BNF tabanlı parserlerin bakımı kolaydır.

Copyright © 2015 Pearson. All rights reserved.

4.1.3 Lexical ve Syntax Analizini Ayrıştırmanın Nedenleri

- *Basitlik* - lexical analiz için daha az karmaşık yaklaşımlar kullanılabilir; bunları ayırmak, parseri basitleştirir.
- *Verimlilik* - ayrıştırma, sözcük çözümleyicinin (lexical analyzer) optimizasyonunu sağlar
- *Taşınabilirlik* - lexical analyzer'ın bazı parçaları taşınabilir olmayabilir, ancak parser her zaman taşınabilirdir.

Copyright © 2015 Pearson. All rights reserved.

4.2 Lexical Analiz

- Lexical analyzer, karakter dizileri (character strings) için bir desen eşleştiricidir.
- Lexical analyzer, parser için bir “front-end”dir.
- Kaynak programda birbirine ait alt dizeleri tanımlar - *lexemes*
  - Lexemes, token adı verilen sözcüksel kategoriyle ilişkili bir karakter deseniyle eşleşir.
  - `sum` bir lexeme’dir; onun token’ı `IDENT` olabilir.

Copyright © 2015 Pearson. All rights reserved.

4.2 Lexical Analiz (Devam Ediyor)

- Lexical analyzer genellikle, bir sonraki token'a ihtiyaç duyduğunda parser tarafından çağrılan bir işlevdir.
- Lexical analyzer oluşturmak için üç yaklaşım:
  - Tokenlerin biçimsel bir tanımını yazın ve bu tanımdan tablo tabanlı bir lexical analyzer oluşturan bir yazılım aracı kullanın.
  - Token'ları açıklayan bir durum diyagramı tasarlayın ve durum diyagramını uygulayan bir program yazın.
  - Token'ları açıklayan bir durum diyagramı tasarlayın ve durum diyagramının tablo odaklı bir uygulamasını elle oluşturun.

Copyright © 2015 Pearson. All rights reserved.

4.2.1 Durum Diyagramı Tasarımı

- İlkel bir durum diyagramı, kaynak dildeki her karakterdeki her durumdan bir geçişe sahip olurdu - böyle bir diyagram çok büyük olurdu!

Copyright © 2015 Pearson. All rights reserved.

4.2.2 Lexical Analiz (Devam Ediyor)

- Çoğu durumda, geçişler durum diyagramını basitleştirmek için birleştirilebilir.
  - Bir tanımlayıcıyı tanırken, tüm büyük ve küçük harfler eşdeğerdir.
    - Tüm harfleri içeren bir karakter sınıfı kullanın
  - Bir integer literali tanırken, tüm rakamlar eşdeğerdir - bir digit sınıfı kullanın.

Copyright © 2015 Pearson. All rights reserved.

4.2.2 Lexical Analiz (Devam Ediyor)

- Rezerve edilmiş kelimeler ve tanımlayıcılar birlikte tanınabilir (her rezerve edilmiş kelime için ayrı bir diyagram parçası yerine).
  - Bir tablo araması (lookup) kullanarak olası bir tanımlayıcının aslında ayrılmış bir kelime olup olmadığını belirleyin.

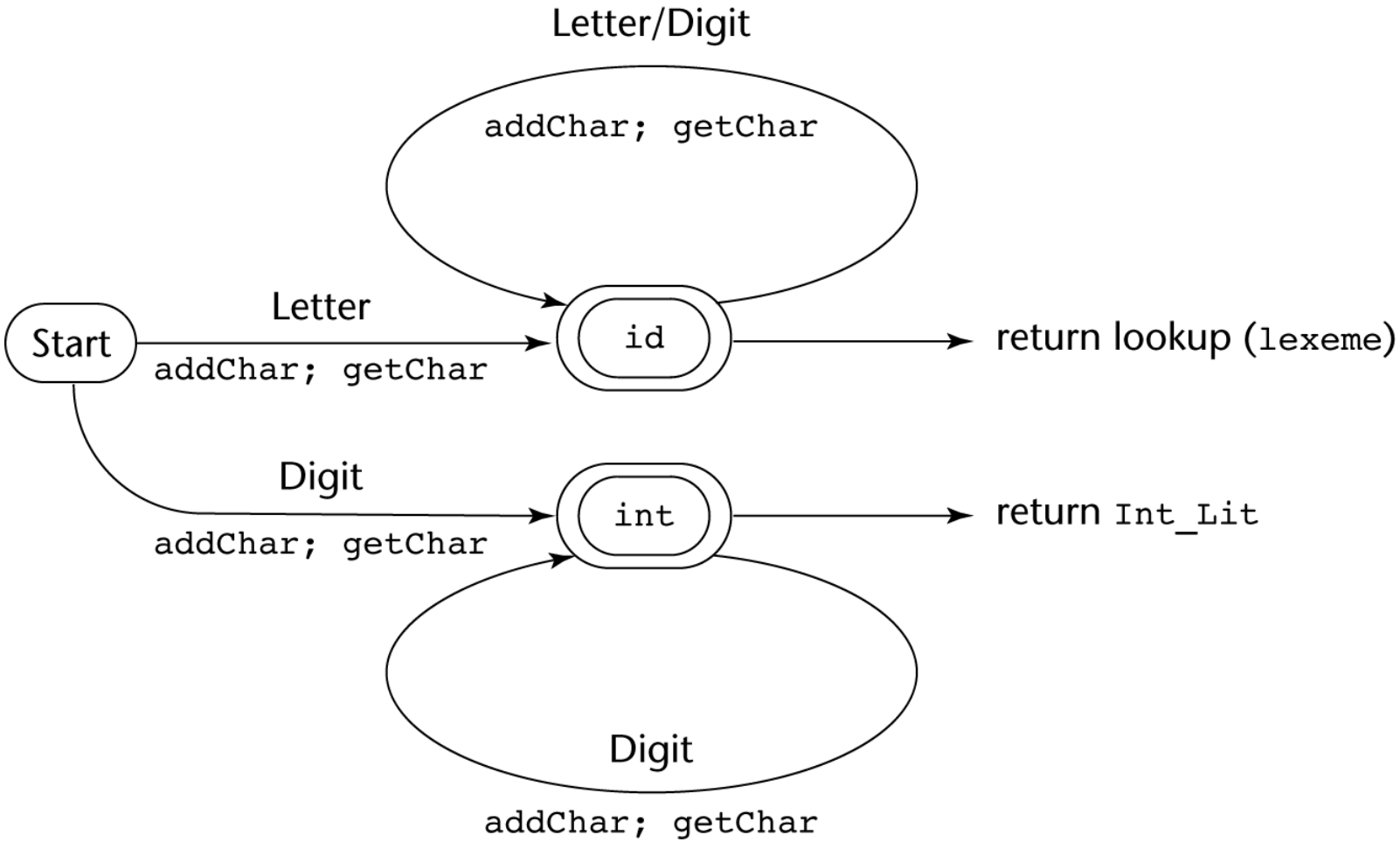
Copyright © 2015 Pearson. All rights reserved.

4.2.2 Lexical Analiz (Devam Ediyor)

- Kullanışlı yardımcı alt programlar:
  - **`getChar`** - girişin bir sonraki karakterini alır, onu nextChar'a koyar, sınıfını belirler ve sınıfı charClass'a koyar
  - **`addChar`** - nextChar karakterini lexeme biriktirilen yere koyar
  - **`lookup`** - lexeme içindeki stringin ayrılmış bir kelime olup olmadığını belirler (bir kod döndürür)

Copyright © 2015 Pearson. All rights reserved.

4.2.3 Durum Diyagramı



Copyright © 2015 Pearson. All rights reserved.

4.2.4 Lexical Analyzer

Uygulama:

→ SHOW `front.c` (pp. 172-177)

- Aşağıda, `(sum + 47) / total` üzerinde kullanıldığında `front.c`'nin lexical analyzer'ının çıktısı verilmiştir.

```
Next token is: 25 Next lexeme is (
Next token is: 11 Next lexeme is sum
Next token is: 21 Next lexeme is +
Next token is: 10 Next lexeme is 47
Next token is: 26 Next lexeme is )
Next token is: 24 Next lexeme is /
Next token is: 11 Next lexeme is total
Next token is: -1 Next lexeme is EOF
```

Copyright © 2015 Pearson. All rights reserved.

4.3 The Parsing Problem

- Giriş programı verildiğinde, parserin hedefleri:
  - Tüm syntax hatalarını bul; her biri için uygun bir tanılama mesajı oluştur ve hızlı bir şekilde düzelt.
  - Program için parse tree'yi veya en azından parse tree'nin izini oluştur.

Copyright © 2015 Pearson. All rights reserved.

4.3 The Parsing Problem (Devam Ediyor)

- İki tür parser vardır:
  - *Top down* - kökten başlayarak parse tree'yi oluşturur
    - Sırası leftmost derivation'dır
    - Preorder olarak parse tree'yi izler veya oluşturur
  - *Bottom up* - yapraklarından başlayarak parse tree'yi oluşturur.
    - Sırası rightmost derivation'ın tersidir
- Yararlı parserlar, girdide sadece bir token ileriye bakar.

Copyright © 2015 Pearson. All rights reserved.

4.3 The Parsing Problem (Devam Ediyor)

- Top-down Parsers
  - Bir cümle biçimi olan xAα verildiğinde, parser, yalnızca A tarafından üretilen ilk belirteci kullanarak, leftmost derivation'da bir sonraki cümle biçimini elde etmek için doğru A kuralını seçmelidir.
- En yaygın Top-down parsing algoritmaları:
  - Recursive descent - kodlanmış bir uygulama
  - LL parsers - tabloya dayalı bir uygulama

Copyright © 2015 Pearson. All rights reserved.

4.3 The Parsing Problem (Devam Ediyor)

- Bottom-up parsers
  - Doğru bir cümle biçimi α verildiğinde, α'nın hangi substringinin, sağ türetmede önceki cümle biçimini üretmek için indirgenmesi gereken gramer kuralının sağ tarafında olduğunu belirler.
  - En yaygın bottom-up parsing algoritmaları LR ailesindedir.

Copyright © 2015 Pearson. All rights reserved.

4.3 The Parsing Problem (Devam Ediyor)

- Parsing'in Karmaşıklığı
  - Herhangi bir belirsiz olmayan (unambiguous) gramer için çalışan parser’lar karmaşık ve verimsizdir ($O(n^3)$, burada n girdinin uzunluğudur).
  - Derleyiciler (Compilers), tüm belirsiz olmayan (unambiguous) gramerlerin yalnızca bir alt kümesi için çalışan, ancak bunu doğrusal zamanda (O(n), burada n girdinin uzunluğudur) yapan parser’lar kullanır.

Copyright © 2015 Pearson. All rights reserved.

4.4 Özyinelemeli İniş Parsing (Recursive-Descent Parsing)

- Gramerdeki her bir nonterminal için, o nonterminal tarafından üretilebilen cümleleri ayrıştırabilen bir alt program vardır.
- EBNF, recursive-descent bir parser için temel oluşturmak için idealdir, çünkü EBNF nonterminal sayısını en aza indirir.

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Basit ifadeler için gramer:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → id | int_constant | ( <expr> )
```

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- `lex` adlı bir lexical analyzer'ımız olduğunu varsayalım, bu analyzer bir sonraki token kodunu `nextToken`'a yerleştirir.
- Tek bir RHS olduğunda kodlama süreci:
  - RHS'deki her terminal sembolünü bir sonraki giriş belirteciyle karşılaştırın; eşleşirlerse devam edin, aksi takdirde bir hata vardır.
  - RHS'deki her bir nonterminal sembol için, ilişkili parsing alt programını çağırın.

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

```
/* Function expr
   Parses strings in the language
   generated by the rule:
   <expr> → <term> {(+ | -) <term>}
 */

void expr() {

/* Parse the first term */

  term();
/* As long as the next token is + or -, call
   lex to get the next token and parse the
   next term */

  while (nextToken == ADD_OP ||
         nextToken == SUB_OP){
    lex();
    term();
  }
}
```

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Bu özel rutin hataları algılamaz.
- Kural: Her parsing rutini bir sonraki token'ı **`nextToken`**'da bırakır.

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Birden fazla RHS içeren bir nonterminal, hangi RHS'yi parse edeceğini belirlemek için bir başlangıç süreci gerektirir.
  - Doğru RHS, bir sonraki girdi tokeni (ileriye bakma) temelinde seçilir.
  - Bir sonraki token, eşleşme bulunana kadar her bir RHS tarafından üretilebilen ilk token ile karşılaştırılır.
  - Eşleşme bulunmazsa, bu bir syntax hatasıdır.

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

```
/* term
Parses strings in the language generated by the rule:
<term> -> <factor> {(* | /) <factor>)
*/
void term() {

/* Parse the first factor */
  factor();

/* As long as the next token is * or /,
   next token and parse the next factor */
  while (nextToken == MULT_OP || nextToken == DIV_OP) {
    lex();
    factor();
  }
} /* End of function term */
```

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

```
/* Function factor
   Parses strings in the language
   generated by the rule:
   <factor> -> id  |  (<expr>)  */

 void factor() {

 /* Determine which RHS */
   if (nextToken) == ID_CODE || nextToken == INT_CODE)

 /* For the RHS id, just call lex */
     lex();

/* If the RHS is (<expr>) - call lex to pass over the left parenthesis,
   call expr, and check for the right parenthesis */
  else if (nextToken == LP_CODE) {
   lex();
      expr();
     if (nextToken == RP_CODE)
        lex();
      else
        error();
    }  /* End of else if (nextToken == ...  */

   else error(); /* Neither RHS matches */
}
```

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- **Trace of the lexical and syntax analyzers on** `(sum + 47) / total`

```
Next token is: 25 Next lexeme is (          Next token is: 11 Next lexeme is total
Enter <expr>                        Enter <factor>
Enter <term>                        Next token is: -1 Next lexeme is EOF
Enter <factor>                              Exit <factor>
Next token is: 11 Next lexeme is sum        Exit <term>
Enter <expr>                        Exit <expr>
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>
Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>
```

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- The LL Grammar Class
  - The Left Recursion Problem
    - Bir gramer doğrudan veya dolaylı olarak sol recursion içeriyorsa, top-down parser için temel oluşturmaz.
      - Bir gramer, doğrudan sol özyinelemeyi kaldırmak için aşağıdaki şekilde değiştirilebilir:

      For each nonterminal, A,

      1. Group the A-rules as $A \rightarrow A\alpha_1 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$ where none of the β's begins with A
      2. Replace the original A-rules with

         $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$

         $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Top-down parsing'i engelleyen gramerlerin diğer bir özelliği, ikili ayrık olmama durumudur.
  - Bir lookahead tokeni temelinde doğru RHS'yi belirleyememe
  - Def: FIRST(α) = {a | α =>* aβ }

    (If α =>* ε, ε is in FIRST(α))

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Çiftler Halinde Ayrık Olma Testi (Pairwise Disjointness Test):
  - Birden fazla RHS içeren gramerdeki her bir nonterminal A için, A → αi ve A → αj kural çiftleri için, aşağıdaki koşulların geçerli olması gerekir:

    $FIRST(\alpha_i) \cap FIRST(\alpha_j) = \phi$
- Örneğin:

  A → a | bB | cAb

  A → a | aB

Copyright © 2015 Pearson. All rights reserved.

4.4 Recursive-Descent Parsing (Devam Ediyor)

- Sol faktörleme (Left factoring) sorunu çözebilir. Replace

<variable> → identifier | identifier [<expression>]

with

<variable> → identifier <new>

<new> → ε | [<expression>]

or

<variable> → identifier [[<expression>]]

(the outer brackets are metasymbols of EBNF)

Copyright © 2015 Pearson. All rights reserved.

4.5 Aşağıdan Yukarıya Parsing (Bottom-Up Parsing)

- Parsing problemi, türetmede önceki doğru cümle biçimini elde etmek için indirgenmesi gereken doğru cümle biçiminde doğru RHS'yi bulmaktır.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Handle hakkında:
  - Def: β is the *handle* of the right sentential form γ = αβw if and only if $S =>^*_{rm} \alpha Aw =>_{rm} \alpha\beta w$
  - Def: β is a *phrase* of the right sentential form γ if and only if $S =>^* \gamma = \alpha_1 A \alpha_2 =>^+ \alpha_1 \beta \alpha_2$
  - Def: β is a *simple phrase* of the right sentential form γ if and only if $S =>^* \gamma = \alpha_1 A \alpha_2 => \alpha_1 \beta \alpha_2$

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Handle hakkında (devam ediyor):
  - Sağ cümle formunun handle'ı, en soldaki basit cümledir.
  - Bir parse tree verildiğinde, artık handle'ı bulmak kolaydır.
  - Parsing, handle budama olarak düşünülebilir.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Shift-Reduce Algorithms
  - Azaltma, parse yığınının üstündeki handle'i karşılık gelen LHS ile değiştirme işlemidir.
  - Shift, bir sonraki token'ı parse yığınının en üstüne taşıma işlemidir.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- LR parser'ların avantajları:
  - Programlama dillerini tanımlayan neredeyse tüm gramerler için çalışacaklardır.
  - Diğer bottom-up algoritmalardan daha geniş bir gramer sınıfında çalışırlar, ancak diğer bottom-up parser'lar kadar verimlidirler.
  - Syntax hatalarını mümkün olan en kısa sürede tespit edebilirler.
  - LR sınıfı gramerler, LL parserleri tarafından ayrıştırılabilen sınıfın bir üst kümesidir.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- LR parserleri bir araçla oluşturulmalıdır.
- Knuth'un görüşü: Bir bottom-up parser, mevcut noktaya kadar olan tüm parse geçmişini kullanarak parse kararları verebilir.
  - Oluşabilecek farklı parse durumlarının sayısı sınırlı ve nispeten azdır, bu nedenle geçmiş, parse yığınında bir parse durumu olarak saklanabilir.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Bir LR yapılandırması, bir LR parserinin durumunu depolar.

$(S_0X_1S_1X_2S_2...X_mS_m, a_ia_{i+1}...a_n\$)$

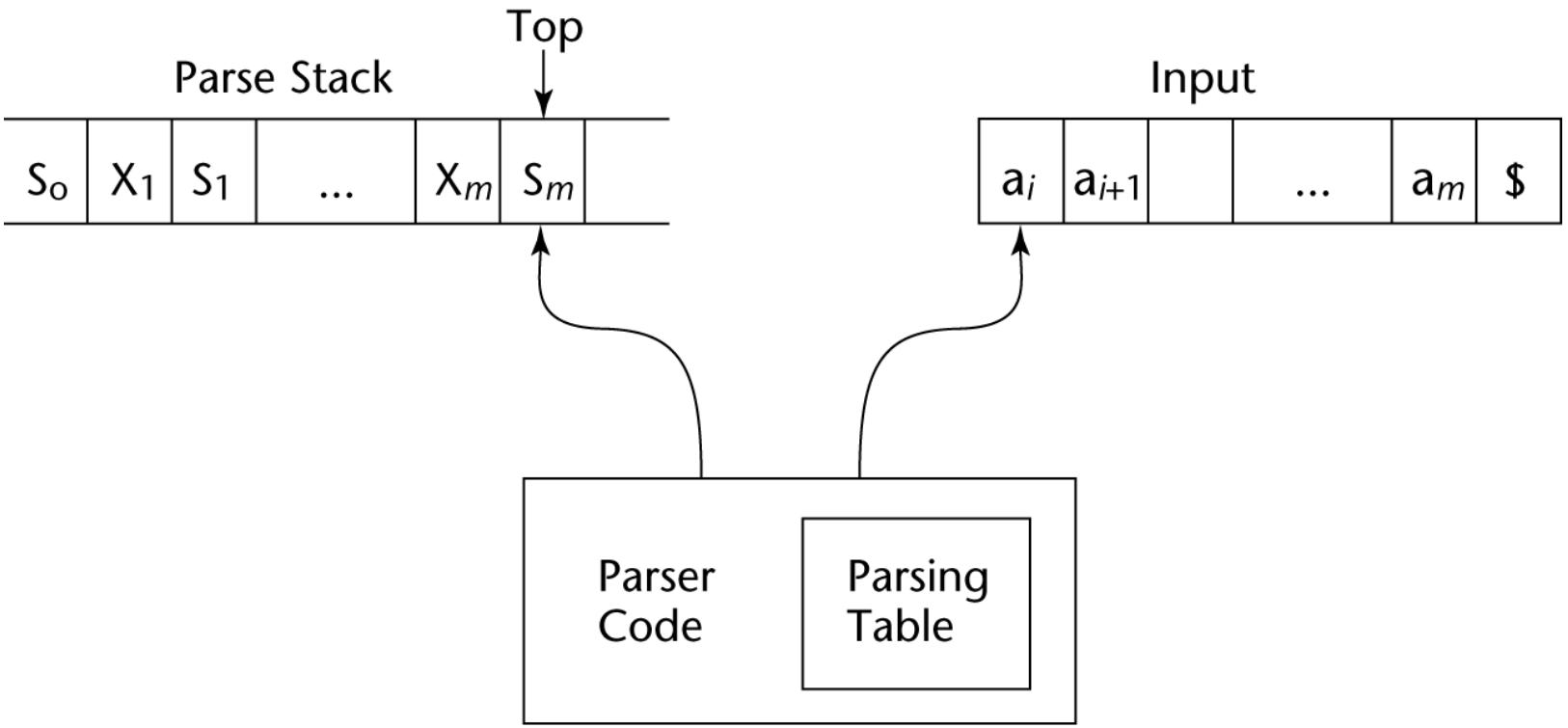
Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- LR parserleri tablo tabanlıdır ve tablo iki bileşene sahiptir: ACTION tablosu ve GOTO tablosu.
  - ACTION tablosu, parser durumu ve bir sonraki token göz önüne alındığında, parser'ın eylemini belirtir.
    - Satırlar durum isimleridir; sütunlar terminallerdir.
  - GOTO tablosu, bir indirgeme eylemi gerçekleştirildikten sonra parse yığınının en üstüne hangi durumun yerleştirileceğini belirtir.
    - Satırlar durum isimleridir; sütunlar nonterminallerdir.

Copyright © 2015 Pearson. All rights reserved.

4.5.1 LR Parser'ın Yapısı



Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- İlk yapılandırma: (S_0, $a_1 \ldots a_n$\$)
- Parser aksiyonları:
  - Shift için, bir sonraki giriş sembolü, Action tablosundaki Shift spesifikasyonunun bir parçası olan durum sembolüyle birlikte yığına itilir.
  - Bir Reduce için, yığından handle'ı durum sembolleriyle birlikte kaldırın. Kuralın LHS'sini itin. Yığındaki yeni LHS'nin hemen altındaki durum sembolünü ve yeni kuralın LHS'sini GOTO tablosuna satır ve sütun olarak kullanarak GOTO tablosundan durum sembolünü itin.

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Parser aksiyonları (devam ediyor):
  - Accept için, parse işlemi tamamlandı ve hata bulunmadı.
  - Bir Error için, parser bir error-handling rutini çağırır.

Copyright © 2015 Pearson. All rights reserved.

4.5.2 LR Parsing Table

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Copyright © 2015 Pearson. All rights reserved.

4.5 Bottom-up Parsing (Devam Ediyor)

- Bir parser table, **`yacc`** veya **`bison`** gibi bir araçla belirli bir gramerden oluşturulabilir.

Copyright © 2015 Pearson. All rights reserved.

4.6 Özet

- Syntax analizi, dil uygulamasının ortak bir parçasıdır
- Lexical Analyzer, bir programın küçük ölçekli parçalarını ayıran bir desen eşleştiricidir
  - Syntax hatalarını tespit eder
  - Bir parse tree üretir
- Recursive-descent parser LL parser'dır
  - EBNF
- Bottom-up parserlar için ayrıştırma sorunu: mevcut cümle formunun alt dizesini bulma
- LR ailesi shift-reduce parserleri, en yaygın bottom-up parsing yaklaşımıdır

Copyright © 2015 Pearson. All rights reserved.